

Relatório de Auditoria de Segurança

— Plushify —

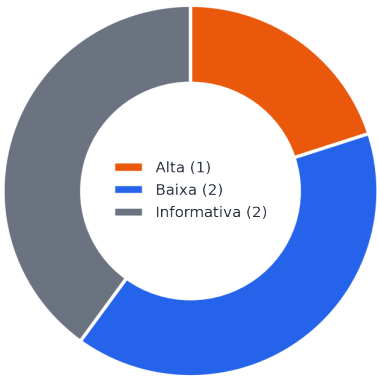
Data da auditoria	29/08/2026
Escopo	Frontend (React/Vite/TS) + Backend Supabase (Postgres/RLS + 19 Edge Functions Deno) 271 migrations · 68 tabelas ativas em public · 116 funções admin_*
Metodologia	5 categorias adaptadas à stack detectada: 1. Banco sem tranca → Row Level Security (RLS) do Postgres/Supabase 2. Permissão no navegador → gates de UI vs. checagem server-side (RPC SECURITY DEFINER / Edge Function) 3. IDOR → posse de recurso validada por JWT em cada handler de Edge Function 4. Chaves expostas → grep de segredos em código, configs e histórico git 5. XSS → dangerouslySetInnerHTML/innerHTML, sanitização e templates de e-mail

Nenhuma falha crítica foi identificada. A superfície auditada (RLS, permissões administrativas, posse de recursos, segredos e sanitização de entrada) mostra padrão defensivo consistente, incluindo evidências de hardening de uma auditoria de bug bounty anterior já incorporada ao código. Um achado de severidade alta foi identificado no modelo de confiança do webhook de pagamento.

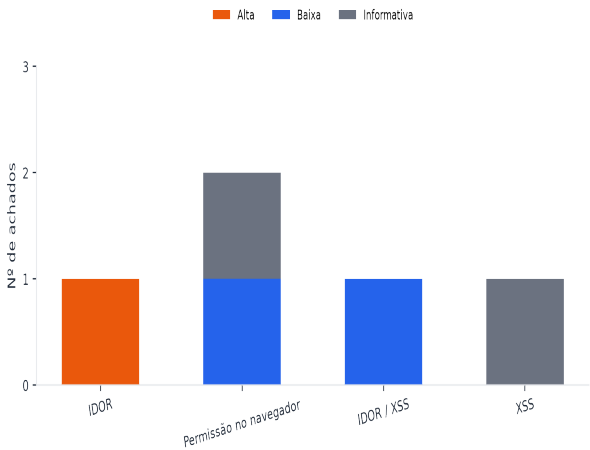
Resumo Executivo

0 Crítica	1 Alta	0 Média	2 Baixa	2 Informativa
---------------------------------	------------------------------	-------------------------------	-------------------------------	-------------------------------------

5 achados no total (0 críticos) e 15 pontos fortes verificados com evidência de código — a auditoria cobriu as 271 migrations e as 19 edge functions por completo, não por amostragem.



Achados por severidade



Achados por categoria

Pontos Fortes

O que foi verificado como protegido, com evidência direta no código — comprova a cobertura real da auditoria.

Categoria	Evidência	Por que está correto
RLS	supabase/migrations (68 tabelas ativas em public)	100% das tabelas vivas em public têm ROW LEVEL SECURITY habilitado. Tabelas financeiras/PII (payments, clients, appointments, financial_transactions etc.) usam policy owner-scoped FOR ALL USING (user_id = auth.uid()) — verificado em 20260608174652_*.sql:446-465.
RLS	20260801040000_enforce_aal2_defense_in_depth.sql:56-70	Policy RESTRICTIVE aplicada dinamicamente a toda tabela com RLS no schema public, exigindo AAL2 (2FA) para quem tem TOTP habilitado — fecha a lacuna de um token AAL1 vazado sendo usado via API direta, contornando o gate de MFA da UI.
RLS	20260814190000_move_prospects_to_internal_schema.sql:9-11	Tabelas de prospecção (prospects, prospect_interactions, prospectors) movidas para o schema internal, fora da API pública do PostgREST — acesso só via 14 RPCs admin_* com checagem de has_role() como primeira linha.
RLS	20260814170000_tighten_grants_bug_bounty.sql	Migration de hardening dedicada (referencia relatório de bug bounty anterior) que revoga TRUNCATE/DELETE/UPDATE desnecessários em audit_logs, commissions, financial_transactions, user_roles, mesmo sem policy que os tornasse exploráveis — redução de superfície defensiva.
RLS	user_roles — 20260728030000_admin_dashboard_foundation.sql:57-78	Tabela de privilégio sem policy de UPDATE (bloqueada por padrão); INSERT/DELETE restritos a has_role(auth.uid(),'admin'). Nenhum caminho de auto-promoção a admin encontrado.
RLS	20260730000000_remove_whatsapp_residue.sql	20 tabelas legadas do módulo WhatsApp abandonado (com policies USING (true), as únicas encontradas em todo o repositório) foram completamente removidas após confirmar que nada em src/ ou edge functions dependia delas.
Permissão no navegador	src/hooks/uselsAdmin.ts:5-29	Hook documentado como uso exclusivo de UI; RPC has_role sempre chamada com o próprio user!.id (auth.uid()), nunca dado sensível extra retornado ao client.
Permissão no navegador	116 funções admin_* nas 271 migrations	Todas, sem exceção, começam com IF NOT public.has_role(auth.uid(),'admin') THEN RAISE EXCEPTION — a checagem de privilégio é sempre no servidor, nunca delegada ao frontend.
Permissão no navegador	abacate-refund-checkout/index.ts:57-66 e abacate-manage-coupons	Checagem de admin feita no servidor com userId vindo de getClaims(token) (JWT verificado), nunca de flag enviado pelo cliente.
IDOR	19/19 edge functions lidas por completo	Padrão consistente: ID do usuário sempre derivado do JWT (getClaims), nunca aceito como parâmetro confiável do cliente. delete-account, whatsapp-proxy, abacate-refund-checkout, start-trial confirmam posse do recurso antes de agir.

Categoria	Evidência	Por que está correto
IDOR	secure-login / secure-signup	Mensagens de erro genéricas anti-enumeração (correção documentada de bug bounty anterior), rate-limit por IP via check_public_rate_limit.
IDOR	abacate-create-checkout / abacate-create-subscription / abacate-create-upgrade-checkout	Preço sempre revalidado contra o catálogo real da AbacatePay no servidor — nunca aceito do cliente diretamente.
Chaves expostas	Repositório completo + git log --all -p	Nenhum .env real no working tree (.gitignore cobre .env/.env.*); zero ocorrências de chave real em código, config, migrations ou histórico git; SERVICE_ROLE_KEY usada só server-side em edge functions, nunca no bundle do frontend.
XSS	src/ (6 ocorrências de dangerouslySetInnerHTML/innerHTML)	Todas com conteúdo estático ou gerado por código do próprio dev (ex: chart.tsx a partir de ChartConfig), nunca input livre de usuário. Nenhum eval()/new Function() encontrado.
XSS	auth-email-hook + _shared/email-templates/*.tsx	E-mails renderizados via React Email (JSX com auto-escape), não pelos .html estáticos (magic-link-email.html etc. usam apenas sintaxe nativa do GoTrue, preenchida pelo próprio Supabase Auth).

Pontos Fracos — Riscos Centrais

ALTA

Webhook de pagamento (AbacatePay) confia só em secret de query string, sem verificação HMAC do payload — limitação documentada do provedor, não bug de implementação.

BAIXA

RPC has_role() aceita _user_id arbitrário, permitindo enumerar quais contas são admin (sem escalar privilégio).

BAIXA

Campo social_link (só editável por admin) renderizado em href sem validar esquema — self-XSS de baixo impacto.

Achados Detalhados

ALTA

F1 · IDOR

Autenticidade do webhook de pagamento depende só de secret estático em query string

supabase/functions/abacate-webhook/index.ts:132-321

```
const receivedSecret = url.searchParams.get('webhookSecret')
if (!receivedSecret || !secretsMatch(receivedSecret, expectedSecret)) { ...401 }
...
let parsed = parseExternalId(externalId) // "plushify:<userId>:<planType>:..."
await admin.rpc('start_subscription', { p_user_id: userId, p_plan_code: planType, ... })
```

Descrição: O webhook da AbacatePay ativa/revoga assinatura paga de um `userId` inteiramente a partir do `external_id`/metadata do payload recebido, sem verificação HMAC de assinatura do corpo da requisição — apenas um segredo estático comparado em tempo constante, mas transmitido via query string (não em header), onde fica mais sujeito a vaziar em logs de proxy/CDN/observabilidade.

Por que é explorável: Quem obtiver o `webhookSecret` pode forjar um POST para ativar (ou revogar) o plano pago de qualquer `userId` à vontade, sem precisar de uma transação real na AbacatePay.

Condições: Depende de exposição do `webhookSecret` (não encontrada vazada nesta auditoria — é `env var`, não `hardcoded` no código). O risco é o modelo de confiança em si: um único segredo em URL, sem assinatura de payload, é uma limitação documentada do provedor AbacatePay, não um bug de implementação do Plushify.

BAIXA

F2 · Permissão no navegador

RPC `has_role` aceita `_user_id` arbitrário — permite enumerar quem é admin

supabase/migrations/20251005144801_*.sql (função `has_role`) +
20260728030000_admin_dashboard_foundation.sql:30-43 / 52-55

```
-- has_role(uuid, app_role) é SECURITY DEFINER e recebe _user_id livre
GRANT EXECUTE ON FUNCTION public.has_role TO authenticated;
-- chamada possível do client:
supabase.rpc('has_role', { _user_id: '<uuid-de-outro-usuário>', _role: 'admin' })
```

Descrição: A função `has_role()` não força `_user_id = auth.uid()`: qualquer usuário autenticado pode consultar se um UUID arbitrário tem papel de admin.

Por que é explorável: Não escala privilégio por si só (todas as 116 RPCs `admin_*` sempre usam `has_role(auth.uid(), 'admin')`, nunca o valor vindo do client) — mas permite enumerar quais UUIDs são administradores da plataforma, útil para preparar phishing/engenharia social direcionada.

Condições: Usuário só precisa estar autenticado; nenhuma outra pré-condição.

BAIXA

F3 · IDOR / XSS

Campo `social_link` renderizado em href sem validar esquema (self-XSS)

src/components/admin/prospects/ProspectDetailDialog.tsx:179

```
<a href={prospect.social_link} target="_blank" rel="noopener noreferrer">
```

Descrição: `social_link` é texto livre gravado só por admins via `admin_create_prospect/admin_update_prospect` (ambos com gate de admin no servidor). O valor é usado direto num atributo href sem checar o esquema da URL.

Por que é explorável: Um admin poderia gravar `javascript:...` no próprio campo e executá-lo na própria sessão ao clicar no link. Não é IDOR real (não afeta outros usuários) — é self-XSS de baixo impacto, mas fácil de corrigir.

Condições: Exige que o próprio admin insira uma URL maliciosa e clique nela.

INFORMATIVA

F4 · Permissão no navegador

Endpoint vestigial sem gate de ação de cobrança relevante

supabase/functions/validate-plan-security/index.ts:—

(endpoint de telemetria/auditoria, escopado ao próprio user.id do JWT)

Descrição: A validação de preço que realmente importa está em abacate-create-checkout/abacate-create-subscription, que revalidam contra o catálogo real da AbacatePay no servidor. Este endpoint não expõe ação sensível.

Por que é explorável: Sem impacto de segurança identificado.

Condições: —

INFORMATIVA

F5 · XSS

innerHTML com mensagem de erro global crua

src/main.tsx:71-78

```
el.innerHTML = `... ${e.message} ...` // handler de erro fatal global
```

Descrição: A mensagem de erro (e.message) é interpolada direto em innerHTML no handler de erro fatal no bootstrap da aplicação.

Por que é explorável: Nenhum caminho identificado nesta auditoria onde input controlado por usuário chegue a essa mensagem de erro — mas por defesa em profundidade, textContent seria mais seguro que innerHTML nesse ponto.

Condições: Exigiria uma exceção cujo .message fosse, de alguma forma, controlável por um atacante.

Recomendações Priorizadas

P1	Adicionar verificação HMAC de assinatura do payload no webhook da AbacatePay	Somar ao secret de query string uma verificação de assinatura do corpo (se o provedor suportar) ou, no mínimo, mover o secret para header e adicionar checagem de idempotência/replay por identificador de evento já processado.
P2	Restringir has_role() para não aceitar _user_id arbitrário de client não-privilegiado	Criar uma RPC pública separada (ex: has_role_self()) que ignora o parâmetro e sempre usa auth.uid(), e restringir has_role(uuid,...) a chamadas server-side/RPCs internas.
P3	Validar esquema de URL antes de renderizar em href (social_link e campos livres similares)	Adicionar uma função utilitária isSafeUrl() que aceite apenas http://https://mailto: e aplicá-la em todo href/src alimentado por campo de texto livre gravado via UI administrativa.
P4	Trocar innerHTML por textContent no handler de erro fatal global (src/main.tsx)	Defesa em profundidade — elimina de vez a possibilidade de HTML injetado nessa mensagem, mesmo que hoje nenhum caminho de exploração tenha sido identificado.
P5	Remover ou proteger o endpoint validate-plan-security se não estiver mais em uso ativo	Reduzir superfície de ataque removendo código vestigial, ou documentar claramente seu propósito atual se ainda for necessário.

Issues para o GitHub

Texto completo em Markdown, pronto para copiar e colar na criação de issues.

Issue 1

```
--- ISSUE 1 ---
### [Segurança] Webhook de pagamento AbacatePay depende só de secret em query string, sem
verificação HMAC do payload

**Labels sugeridas:** `security`, `alta`

**Descrição do problema**
O endpoint `supabase/functions/abacate-webhook/index.ts` autentica requisições do provedor de
pagamento
AbacatePay comparando um `webhookSecret` recebido via query string (`?webhookSecret=...`) contra
um
segredo esperado, usando comparação em tempo constante. Não há verificação de assinatura HMAC do
corpo
da requisição. O `userId` que ativa/revoga a assinatura paga vem do `external_id`/metadata do
próprio
payload recebido, sem correlação com um registro pré-existente no lado do Plushify.

**Por que é explorável**
Um segredo em query string está mais sujeito a vazar em logs de proxy reverso, CDN, ou ferramentas
de
observabilidade do que um segredo em header. Se o `webhookSecret` vazar, é possível forjar um POST
para
ativar ou revogar o plano pago de qualquer `userId`, sem uma transação real ocorrida na
AbacatePay.

**Evidência**
`supabase/functions/abacate-webhook/index.ts:132-321`
```ts
const receivedSecret = url.searchParams.get('webhookSecret')
if (!receivedSecret || !secretsMatch(receivedSecret, expectedSecret)) { ...401 }
...
let parsed = parseExternalId(externalId) // "plushify:<userId>:<planType>:<billingPeriod>:<uuid>"
await admin.rpc('start_subscription', { p_user_id: userId, p_plan_code: planType, ... })
```

**Impacto**
Ativação ou revogação fraudulenta de assinatura paga para qualquer usuário, caso o segredo do
webhook
vaze. Não há evidência de vazamento nesta auditoria – o risco é estrutural do modelo de confiança.

**Sugestão de correção**
- Verificar se a AbacatePay suporta assinatura HMAC do payload e implementar essa verificação
adicionalmente ao secret.
- Mover o secret para um header de requisição em vez de query string.
- Adicionar checagem de idempotência/replay por identificador único do evento já processado.
- Rotacionar o `webhookSecret` atual por precaução.

**Critérios de aceite**
- [ ] Verificação de assinatura de payload implementada (HMAC ou equivalente suportado pelo
provedor)
- [ ] Secret movido de query string para header
- [ ] Checagem de replay/idempotência por ID de evento adicionada
- [ ] `webhookSecret` rotacionado em produção
- [ ] Teste automatizado cobrindo rejeição de payload com assinatura inválida
--- FIM ISSUE 1 ---
```

Issue 2

```

--- ISSUE 2 ---
### [Segurança] RPC has_role() permite enumerar quais usuários são admin

**Labels sugeridas:** `security`, `baixa`

**Descrição do problema**
A função `has_role(uuid, app_role)` é `SECURITY DEFINER` e aceita um `_user_id` livre, sem forçar `auth.uid()`. Qualquer usuário autenticado pode chamar `supabase.rpc('has_role', { _user_id: '<uuid de outro usuário>', _role: 'admin' })` e descobrir se aquele UUID tem papel de administrador.

**Por que é explorável**
Não escala privilégio por si só – todas as 116 RPCs `admin_` sempre usam `has_role(auth.uid(), 'admin')`, nunca um valor vindo do cliente. Mas permite enumerar quais contas são administradoras da plataforma, útil para preparar ataques de phishing/engenharia social direcionados a essas contas específicas.

**Evidência**
`supabase/migrations/20251005144801_*.sql:30-43` (definição) e
`supabase/migrations/20260728030000_admin_dashboard_foundation.sql:52-55` (grant)
```sql
GRANT EXECUTE ON FUNCTION public.has_role(uuid, app_role) TO authenticated;
-- has_role() não força _user_id = auth.uid()
```

**Impacto**
Enumeração de contas administrativas por qualquer usuário autenticado – informação útil para engenharia social/phishing direcionado, sem escalar privilégio diretamente.

**Sugestão de correção**
Criar uma RPC pública separada (ex: `has_role_self(app_role)`) que ignora qualquer parâmetro de ID e sempre usa `auth.uid()` internamente; restringir `has_role(uuid, app_role)` para ser chamável apenas por RPCs internas/server-side (revogar `EXECUTE` de `authenticated` na assinatura com `_user_id` livre).

**Critérios de aceite**
- [ ] Nova RPC `has_role_self()` criada e usada pelo frontend (useIsAdmin) em vez da versão irrestrita
- [ ] `EXECUTE` de `has_role(uuid, app_role)` revogado de `authenticated`/`anon`
- [ ] Todas as chamadas internas (RPCs admin_) continuam funcionando sem alteração de comportamento
- [ ] Teste confirmando que um usuário não-admin não consegue mais consultar o papel de outro UUID
--- FIM ISSUE 2 ---

```

Issue 3

```

--- ISSUE 3 ---
### [Segurança] Campo social_link renderizado em href sem validar esquema (self-XSS) + hardening geral de defesa em profundidade

**Labels sugeridas:** `security`, `baixa`

**Descrição do problema**
Esta issue agrupa dois achados triviais relacionados a sanitização de saída, de baixo risco individual:

1. `src/components/admin/prospects/ProspectDetailDialog.tsx:179` renderiza `prospect.social_link` (campo de texto livre, editável só por admins) direto em um atributo `href`, sem validar o esquema da URL. Um admin poderia gravar `javascript:...` no próprio campo e executá-lo na própria sessão.
2. `src/main.tsx:71-78` interpola `e.message` cru via `innerHTML` no handler de erro fatal global. Nenhum caminho de exploração foi identificado (a mensagem não é alimentada por input de usuário em nenhum fluxo mapeado), mas o padrão é frágil por natureza.

```

****Por que é explorável****

(1) é self-XSS – exige que o próprio admin insira e clique numa URL maliciosa; não afeta outros usuários. (2) hoje não tem caminho de exploração conhecido; é hardening preventivo.

****Evidência****

```
``tsx
// src/components/admin/prospects/ProspectDetailDialog.tsx:179
<a href={prospect.social_link} target="_blank" rel="noopener noreferrer">

// src/main.tsx:71-78
el.innerHTML = `... ${e.message} ...`
``
```

****Impacto****

Baixo em ambos os casos – nem um nem outro afeta usuários além do próprio autor da ação, no estado atual do código.

****Sugestão de correção****

- Criar uma função utilitária `isSafeUrl(url)` que aceite apenas esquemas `http:`, `https:` e `mailto:`, e aplicá-la em todo `href`/`src` alimentado por campo de texto livre gravado via UI administrativa (começando por `social_link`).
- Trocar `innerHTML` por `textContent` no handler de erro fatal global em `src/main.tsx`.

****Critérios de aceite****

- [] `isSafeUrl()` criada e aplicada em `ProspectDetailDialog.tsx` (e outros pontos análogos, se existirem)
 - [] `src/main.tsx` usa `textContent` em vez de `innerHTML` para a mensagem de erro
 - [] Teste cobrindo rejeição de `javascript:` em `social_link`
- FIM ISSUE 3 ---